Unidad 11 · Subconsultas y vistas

Qué vas a poder hacer. Responder una pregunta en dos pasos metiendo una consulta dentro de otra (como valor, como lista o como tabla), elegir entre `IN`, `EXISTS` y una subconsulta correlacionada, ordenar una consulta larga con `WITH`, combinar resultados con `UNION`, `INTERSECT` y `EXCEPT`, y guardar una consulta frecuente como vista, sabiendo cuándo se puede escribir a través de ella.

Lo que necesitas antes. La unidad 9 (`SELECT`, `WHERE`, `ORDER BY`, `NULL` e `IS NULL`) y la 10 (`JOIN`, `LEFT JOIN`, `GROUP BY`, `HAVING` y las funciones de agregación `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`). De la unidad 5, qué es una clave foránea.

Tiempo. Unas 6 horas: 2,5 de lectura con SQLite abierto y 3,5 de práctica.

Cómo correr todo. Los scripts están en `sql/`. `u11_subconsultas.sql` crea la base del ejemplo y tiene cada consulta de la unidad; `u11_caso.sql` trae las del caso y se corre sobre una **copia** de `datos/estampa_sur.db`, porque crea vistas. Cada resultado de esta unidad es el que devolvió SQLite 3.50.4; el verificador `verificacion/u11_subconsultas.py` los vuelve a obtener y los busca en el texto.

---

1. La idea: el resultado de una consulta es una tabla

Un `SELECT` devuelve una tabla. Esa tabla se puede usar donde SQL espera un valor, una lista o una tabla. Eso es una **subconsulta**: un `SELECT` entre paréntesis dentro de otra sentencia.

Sirve cuando la pregunta tiene dos pasos y el segundo depende del primero. «¿Qué préstamos duraron más que el promedio?» pide primero el promedio y después compararlo. En una planilla harías una celda con el promedio y una columna que compara contra ella; en SQL la celda es la subconsulta.

Dónde va la subconsulta	Qué tiene que devolver	Ejemplo
en `WHERE`, con `=`, `>` o `<`	**un valor** (1 fila, 1 columna)	`dias > (SELECT AVG(dias) FROM prestamo)`
en `WHERE`, con `IN` o `NOT IN`	**una lista** (1 columna)	`id_socio IN (SELECT id_socio FROM prestamo)`
en `WHERE`, con `EXISTS`	**filas o ninguna** (las columnas no importan)	`EXISTS (SELECT 1 FROM prestamo p WHERE ...)`
en `FROM` o en `WITH`	**una tabla** con nombre	`FROM (SELECT ...) AS x`
en `SELECT`	un valor por fila	`(SELECT COUNT(*) FROM ...) AS prestamos`

La base del ejemplo. Una biblioteca comunal de Tomé presta libros a socios de Tomé, Penco y Coelemu. Cada socio puede haber llegado invitado por otro socio (`invitado_por`); `dias` es cuántos días tuvo el libro, y queda en `NULL` mientras no lo devuelve. Los libros son reales; los socios y los préstamos, inventados.

```
CREATE TABLE socio (
    id_socio     INTEGER PRIMARY KEY,
    nombre       VARCHAR(40) NOT NULL,
    comuna       VARCHAR(30) NOT NULL,
    invitado_por INTEGER REFERENCES socio(id_socio)   -- NULL: nadie lo invitó
);
CREATE TABLE libro (
    id_libro INTEGER PRIMARY KEY,
    titulo   VARCHAR(60) NOT NULL,
    autor    VARCHAR(40) NOT NULL,
    genero   VARCHAR(20) NOT NULL,
    anio     INTEGER NOT NULL
);
CREATE TABLE prestamo (
    id_prestamo INTEGER PRIMARY KEY,
    id_socio    INTEGER NOT NULL REFERENCES socio(id_socio),
    id_libro    INTEGER NOT NULL REFERENCES libro(id_libro),
    fecha       DATE NOT NULL,
    dias        INTEGER                              -- NULL: todavía no lo devuelve
);
```

Los datos, tal como los devuelve cada tabla:

```
SELECT * FROM socio;
```

id_socio	nombre	comuna	invitado_por
1	Ana Soto	Tomé	NULL
2	Benjamín Rojas	Tomé	1
3	Carla Muñoz	Penco	1
4	Diego Fuentes	Tomé	NULL
5	Elisa Vera	Penco	3
6	Felipe Araya	Coelemu	NULL
7	Gabriela Paz	Tomé	4
8	Hugo Leiva	Coelemu	6

```
SELECT * FROM libro;
```

id_libro	titulo	autor	genero	anio
1	Hijo de ladrón	Manuel Rojas	novela	1951
2	Veinte poemas de amor y una canción desesperada	Pablo Neruda	poesía	1924
3	Altazor	Vicente Huidobro	poesía	1931
4	La casa de los espíritus	Isabel Allende	novela	1982
5	Martín Rivas	Alberto Blest Gana	novela	1862
6	Desolación	Gabriela Mistral	poesía	1922
7	Papelucho	Marcela Paz	infantil	1947
8	Mala onda	Alberto Fuguet	novela	1991
9	Poemas y antipoemas	Nicanor Parra	poesía	1954

```
SELECT * FROM prestamo;
```

id_prestamo	id_socio	id_libro	fecha	dias
1	1	1	2026-09-01	12
2	1	2	2026-09-03	5
3	2	4	2026-09-04	20
4	3	2	2026-09-05	7
5	3	5	2026-09-08	15
6	1	6	2026-09-10	9
7	5	7	2026-09-11	4
8	2	1	2026-09-12	NULL
9	7	4	2026-09-15	18
10	3	9	2026-09-16	6
11	1	8	2026-09-18	NULL
12	7	6	2026-09-20	10
13	5	2	2026-09-22	8
14	8	7	2026-09-24	3
15	3	4	2026-09-25	NULL

2. Subconsulta escalar: un solo valor

Una subconsulta **escalar** devuelve una fila y una columna, y se usa como si fuera un número escrito a mano. Primero, el promedio solo:

```
SELECT AVG(dias) AS promedio FROM prestamo;
```

promedio
9.75

`AVG` ignora los `NULL`: promedia los 12 préstamos devueltos (117 días en total) y deja fuera los 3 que siguen afuera. Por eso da 9,75 y no 117/15. Ahora la pregunta completa:

```
SELECT id_prestamo, id_socio, dias
FROM prestamo
WHERE dias > (SELECT AVG(dias) FROM prestamo)
ORDER BY dias DESC;
```

id_prestamo	id_socio	dias
3	2	20
9	7	18
5	3	15
1	1	12
12	7	10

En qué orden lo evalúa el motor. Primero la subconsulta (una sola vez, porque no depende de la fila): 9,75. Después el `FROM`, el `WHERE` fila por fila contra 9,75 (las filas con `dias` en `NULL` no pasan: `NULL > 9.75` no es verdadero) y al final el `ORDER BY`.

Por qué no se puede escribir directo. La intuición dice `WHERE dias > AVG(dias)`. No funciona, porque el `WHERE` mira **una fila a la vez** y el promedio necesita todas las filas:

```
SELECT id_prestamo, dias
FROM prestamo
WHERE dias > AVG(dias);
```

```
misuse of aggregate function AVG()
```

Una función de agregación va en el `SELECT` o en el `HAVING` de una consulta agrupada, nunca en el `WHERE`. Cuando hace falta comparar cada fila con un agregado, el agregado va en una subconsulta.

Si la subconsulta devuelve más de una fila. Con `=` el motor espera un solo valor. Esta consulta quiere «los libros publicados el mismo año que un libro de poesía», pero hay cuatro de poesía:

```
SELECT titulo
FROM libro
WHERE anio = (SELECT anio FROM libro WHERE genero = 'poesía');
```

titulo
Veinte poemas de amor y una canción desesperada

SQLite no reclama: toma **la primera fila** de la subconsulta (1924) y sigue. MySQL, en cambio, se detiene con `ERROR 1242 (21000): Subquery returns more than 1 row`. El error de MySQL es el comportamiento sano: si la subconsulta puede devolver varias filas, la pregunta pedía `IN`.

---

3. Subconsulta de lista: `IN` y `NOT IN`, y la trampa del `NULL`

Con `IN`, la subconsulta devuelve una columna y la condición pregunta si el valor está en esa lista. «¿Qué socios han pedido algún libro de poesía?» se arma de adentro hacia afuera: los libros de poesía, los socios que pidieron esos libros, sus nombres.

```
SELECT nombre
FROM socio
WHERE id_socio IN (SELECT id_socio
                   FROM prestamo
                   WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'poesía'))
ORDER BY nombre;
```

nombre
Ana Soto
Carla Muñoz
Elisa Vera
Gabriela Paz

La lista interna puede repetir valores (Ana pidió dos libros de poesía); `IN` no se ve afectado, y por eso Ana sale una vez. Con un `JOIN` habría salido dos veces y habría hecho falta un `DISTINCT`.

`NOT IN` pregunta lo contrario. Los socios que nunca han pedido un libro:

```
SELECT nombre
FROM socio
WHERE id_socio NOT IN (SELECT id_socio FROM prestamo)
ORDER BY nombre;
```

nombre
Diego Fuentes
Felipe Araya

La trampa. Ahora, con la misma forma, los socios que **no han invitado a nadie**: los que no aparecen en la columna `invitado_por`. Esa columna tiene `NULL`:

```
SELECT invitado_por FROM socio;
```

invitado_por
NULL
1
1
NULL
3
NULL
4
6

```
SELECT nombre
FROM socio
WHERE id_socio NOT IN (SELECT invitado_por FROM socio);
```

nombre

Ninguna fila, aunque Benjamín, Elisa, Gabriela y Hugo no han invitado a nadie. La razón está en cómo SQL trata el `NULL`: no es un valor, es «no se sabe». `2 NOT IN (NULL, 1, 1, NULL, 3, ...)` se traduce en una cadena de `AND`:

$$2 \neq \text{NULL} \ \wedge \ 2 \neq 1 \ \wedge \ 2 \neq 3 \ \wedge \ \ldots \ = \ \text{desconocido} \wedge \text{verdadero} \wedge \ldots \ = \ \text{desconocido}$$

¿Es 2 distinto de un valor que no se conoce? No se sabe. Y el `WHERE` deja pasar solo lo **verdadero**: lo desconocido se descarta igual que lo falso. Con un solo `NULL` en la lista, `NOT IN` no deja pasar nada. (`IN` no sufre esto: basta que el valor coincida con uno para que sea verdadero.)

Expresión	Resultado	¿Pasa el `WHERE`?
`2 IN (1, 2, NULL)`	verdadero	sí
`5 IN (1, 2, NULL)`	desconocido	no
`5 NOT IN (1, 2)`	verdadero	sí
`5 NOT IN (1, 2, NULL)`	desconocido	**no**

Hay dos arreglos. El primero saca los `NULL` de la lista:

```
SELECT nombre
FROM socio
WHERE id_socio NOT IN (SELECT invitado_por FROM socio WHERE invitado_por IS NOT NULL)
ORDER BY nombre;
```

nombre
Benjamín Rojas
Elisa Vera
Gabriela Paz
Hugo Leiva

El segundo, que es el que conviene acostumbrarse a usar, es `NOT EXISTS` (sección 5). La regla práctica: **antes de escribir `NOT IN (SELECT col ...)`, mira si `col` admite `NULL`.** Si es una clave primaria o tiene `NOT NULL`, no hay riesgo; si no, usa `NOT EXISTS`.

---

4. Subconsulta correlacionada

Hasta acá cada subconsulta se podía correr sola. Una subconsulta **correlacionada** usa una columna de la consulta de afuera, y por eso su resultado cambia con cada fila. «El libro más reciente de cada género»:

```
SELECT genero, titulo, anio
FROM libro l
WHERE anio = (SELECT MAX(l2.anio) FROM libro l2 WHERE l2.genero = l.genero)
ORDER BY genero;
```

genero	titulo	anio
infantil	Papelucho	1947
novela	Mala onda	1991
poesía	Poemas y antipoemas	1954

Cómo se lee. Para cada fila `l` de afuera, el motor calcula el máximo de los libros `l2` **de su mismo género** (`l2.genero = l.genero`) y deja la fila si su año es ese máximo. Para *Hijo de ladrón* (novela, 1951) el máximo de las novelas es 1991: no pasa. Para *Mala onda*, sí. Los alias `l` y `l2` son obligatorios: la misma tabla aparece dos veces y hay que decir cuál es cuál.

Conceptualmente la subconsulta corre una vez por fila de afuera; el motor suele optimizarlo, pero la lectura «una vez por fila» es la que permite escribirla bien.

Correlacionada en el `SELECT`. También puede calcular una columna. Los préstamos de cada socio:

```
SELECT s.nombre,
       (SELECT COUNT(*) FROM prestamo p WHERE p.id_socio = s.id_socio) AS prestamos
FROM socio s
ORDER BY s.id_socio;
```

nombre	prestamos
Ana Soto	4
Benjamín Rojas	2
Carla Muñoz	4
Diego Fuentes	0
Elisa Vera	2
Felipe Araya	0
Gabriela Paz	2
Hugo Leiva	1

Diego y Felipe aparecen con 0. Un `JOIN` con `GROUP BY` los habría perdido (no tienen préstamos con qué juntarse); para conservarlos haría falta un `LEFT JOIN` con `COUNT(p.id_prestamo)`, como en la unidad 10.

---

5. `EXISTS` y `NOT EXISTS`

`EXISTS (subconsulta)` es verdadero si la subconsulta devuelve **al menos una fila**, sin importar qué columnas. Por costumbre se escribe `SELECT 1`. Casi siempre es correlacionada. Los libros que nunca se han prestado:

```
SELECT titulo
FROM libro l
WHERE NOT EXISTS (SELECT 1 FROM prestamo p WHERE p.id_libro = l.id_libro);
```

titulo
Altazor

Y los socios que no han invitado a nadie, ahora sin trampa:

```
SELECT nombre
FROM socio s
WHERE NOT EXISTS (SELECT 1 FROM socio i WHERE i.invitado_por = s.id_socio)
ORDER BY nombre;
```

nombre
Benjamín Rojas
Elisa Vera
Gabriela Paz
Hugo Leiva

Por qué `NOT EXISTS` no cae en la trampa. No compara `s.id_socio` contra una lista: pregunta si hay alguna fila que cumpla `i.invitado_por = s.id_socio`. Las filas con `invitado_por` en `NULL` simplemente no cumplen esa condición y no cuentan. El resultado es siempre verdadero o falso, nunca desconocido.

Quiero	Escribo
que el valor esté en una lista corta y fija	`IN ('novela', 'poesía')`
que exista una fila relacionada	`EXISTS` o `IN (SELECT ...)`: dan lo mismo
que **no** exista una fila relacionada	`NOT EXISTS` (o `NOT IN` solo si la columna es `NOT NULL`)
comparar con un agregado de su propio grupo	subconsulta correlacionada

6. Subconsultas en el `FROM`: tablas derivadas

Una subconsulta en el `FROM` es una **tabla derivada**: existe solo mientras corre la consulta y se usa como cualquier tabla. Sirve para **agregar dos veces**, algo que un solo `GROUP BY` no puede hacer. «¿Qué socios piden más que el socio promedio?» necesita primero contar por socio y después promediar esos conteos:

```
SELECT AVG(y.n) AS promedio
FROM (SELECT COUNT(*) AS n FROM prestamo GROUP BY id_socio) AS y;
```

promedio
2.5

```
SELECT x.id_socio, x.n
FROM (SELECT id_socio, COUNT(*) AS n FROM prestamo GROUP BY id_socio) AS x
WHERE x.n > (SELECT AVG(y.n)
             FROM (SELECT COUNT(*) AS n FROM prestamo GROUP BY id_socio) AS y)
ORDER BY x.id_socio;
```

id_socio	n
1	4
3	4

En qué orden lo evalúa el motor. (1) La tabla derivada `x`: un conteo por socio que ha pedido algo (6 filas). (2) La subconsulta escalar: el promedio de esos 6 conteos, 15/6 = 2,5. (3) El `WHERE` deja las filas de `x` sobre 2,5. (4) El `ORDER BY`.

¿Promedio de quiénes? El promedio es de los 6 socios que han pedido algo, no de los 8. Diego y Felipe no están en `x` porque no tienen filas en `prestamo`. Si la pregunta fuera «sobre el promedio de todos los socios», habría que partir de `socio` con `LEFT JOIN` (promedio 15/8 = 1,875) y entrarían también los que piden 2. Una subconsulta no decide eso por ti: decide la tabla de la que partes.

MySQL exige el alias. Toda tabla derivada necesita nombre (`AS x`); sin él, MySQL responde `ERROR 1248 (42000): Every derived table must have its own alias`. SQLite lo acepta sin alias, y por eso conviene ponerlo siempre: así el mismo script corre en los dos.

7. `WITH`: ponerle nombre a una subconsulta

La consulta anterior escribe dos veces la misma tabla derivada. `WITH` (una *expresión de tabla común*, CTE por su sigla en inglés) la declara una vez, con nombre, **antes** de la consulta:

```
WITH por_socio AS (
    SELECT id_socio, COUNT(*) AS n
    FROM prestamo
    GROUP BY id_socio
)
SELECT s.nombre, ps.n
FROM por_socio ps
JOIN socio s ON s.id_socio = ps.id_socio
WHERE ps.n > (SELECT AVG(n) FROM por_socio)
ORDER BY s.nombre;
```

nombre	n
Ana Soto	4
Carla Muñoz	4

Es la misma respuesta (los socios 1 y 3), pero se lee de arriba abajo como una receta: primero armo `por_socio`, después la uso. Se pueden declarar varias separadas por coma (`WITH a AS (...), b AS (...)`), y cada una puede usar las anteriores. La CTE existe solo para esa consulta; no queda guardada (para eso están las vistas, sección 9).

`WITH` existe en SQLite desde la versión 3.8.3 y en MySQL desde la 8.0. En MySQL 5.7 hay que volver a la tabla derivada.

---

8. `UNION`, `INTERSECT` y `EXCEPT`

Estos operadores combinan **dos consultas con las mismas columnas**, como en el álgebra relacional de la unidad 7: unión (∪), intersección (∩) y diferencia (−). Socios que pidieron novela y también poesía:

```
SELECT id_socio FROM prestamo
WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'novela')
INTERSECT
SELECT id_socio FROM prestamo
WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'poesía')
ORDER BY id_socio;
```

id_socio
1
3
7

Los que pidieron novela pero **nunca** poesía:

```
SELECT id_socio FROM prestamo
WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'novela')
EXCEPT
SELECT id_socio FROM prestamo
WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'poesía');
```

id_socio
2

Los que pidieron una cosa u otra:

```
SELECT id_socio FROM prestamo
WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'novela')
UNION
SELECT id_socio FROM prestamo
WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'poesía')
ORDER BY id_socio;
```

id_socio
1
2
3
5
7

`UNION` elimina duplicados; `UNION ALL` no. Las dos consultas juntas tienen 13 filas (7 préstamos de novela y 6 de poesía); `UNION` deja 5 socios distintos. `UNION ALL` es más rápido (no tiene que buscar repetidos) y es el que corresponde cuando después vas a sumar o contar:

```
SELECT COUNT(*) AS filas
FROM (SELECT id_socio FROM prestamo
      WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'novela')
      UNION ALL
      SELECT id_socio FROM prestamo
      WHERE id_libro IN (SELECT id_libro FROM libro WHERE genero = 'poesía'));
```

filas
13

Las reglas. Las dos consultas deben tener el mismo número de columnas, de tipos compatibles; los nombres del resultado son los de la primera. El `ORDER BY` va una sola vez, al final, y ordena el resultado combinado. Si las columnas no calzan:

```
SELECT id_socio, nombre FROM socio
UNION
SELECT id_socio FROM prestamo;
```

```
SELECTs to the left and right of UNION do not have the same number of result columns
```

MySQL. `UNION` y `UNION ALL` existen hace mucho; `INTERSECT` y `EXCEPT` llegaron recién en la 8.0.31. Antes se escriben con `IN` (intersección) y `NOT IN` o `NOT EXISTS` (diferencia).

9. Vistas

Una **vista** es una consulta guardada con nombre. Se consulta como una tabla, pero **no guarda datos**: cada vez que alguien la usa, el motor vuelve a correr su consulta sobre las tablas de ese momento.

```
CREATE VIEW v_prestamo AS
SELECT p.id_prestamo, s.nombre, s.comuna, l.titulo, l.genero, p.fecha, p.dias
FROM prestamo p
JOIN socio s ON s.id_socio = p.id_socio
JOIN libro l ON l.id_libro = p.id_libro;
```

Desde ahora, el `JOIN` de tres tablas se escribe una vez y las preguntas se hacen sobre la vista:

```
SELECT genero, COUNT(*) AS prestamos, AVG(dias) AS dias_promedio
FROM v_prestamo
GROUP BY genero
ORDER BY prestamos DESC;
```

genero	prestamos	dias_promedio
novela	7	16.25
poesía	6	7.5
infantil	2	3.5

La vista está viva. Si entra un préstamo nuevo, la vista lo muestra sin hacer nada. Para no ensuciar la base, la prueba va dentro de una transacción que después se deshace (la unidad 12 explica `BEGIN` y `ROLLBACK`):

```
BEGIN;
```

```
INSERT INTO prestamo VALUES (16, 6, 3, '2026-09-29', NULL);
```

```
SELECT COUNT(*) AS filas FROM v_prestamo;
```

filas
16

```
ROLLBACK;
```

```
SELECT COUNT(*) AS filas FROM v_prestamo;
```

filas
15

Para qué sirven.

- **Escribir una vez un `JOIN` o una regla.** Si «venta» es `cantidad * precio_unit`, que lo diga una vista y no veinte consultas copiadas: cuando la regla cambie, se cambia en un lugar.

- **Simplificar para otros.** La encargada de la biblioteca consulta `v_prestamo` sin saber que hay tres tablas detrás.
- **Permisos.** En un motor con usuarios (MySQL, PostgreSQL), se puede dar permiso sobre la vista y no sobre la tabla: `GRANT SELECT ON v_prestamo TO ...` deja ver los préstamos sin ver otras columnas de `socio` (un RUT, un teléfono). SQLite no tiene usuarios.

¿Se puede escribir a través de una vista? Depende de la vista y del motor.

```
CREATE VIEW v_poesia AS
SELECT id_libro, titulo, anio
FROM libro
WHERE genero = 'poesía';
```

```
UPDATE v_poesia SET titulo = 'Altazor o el viaje en paracaídas' WHERE id_libro = 3;
```

```
cannot modify v_poesia because it is a view
```

En SQLite las vistas son de solo lectura. (Se puede hacer que acepten escritura con *triggers* `INSTEAD OF`, que este ramo no cubre.) **En MySQL**, ese mismo `UPDATE` funciona, porque la vista es **actualizable**: cada fila de la vista corresponde a una sola fila de una tabla, y el motor sabe a cuál escribir. Una vista deja de ser actualizable cuando esa correspondencia se pierde:

La vista tiene	¿Actualizable en MySQL?	Por qué
una tabla, columnas tal cual, un `WHERE`	sí	cada fila es una fila de la tabla
`GROUP BY`, `SUM`, `COUNT`, `DISTINCT` o `HAVING`	no	una fila de la vista resume muchas: ¿a cuál escribir?
`UNION`	no	la fila puede venir de cualquiera de las dos consultas
una columna calculada (`cantidad * precio_unit`)	esa columna no	no hay dónde guardar el producto
un `JOIN`	a veces: solo columnas de una de las tablas	MySQL lo permite con restricciones

Para insertar por la vista, además, las columnas `NOT NULL` sin valor por defecto de la tabla tienen que estar en la vista: `v_poesia` no tiene `autor` ni `genero`, y un `INSERT` por ella fallaría también en MySQL.

`WITH CHECK OPTION` **(MySQL).** `CREATE VIEW v_poesia AS ... WHERE genero = 'poesía' WITH CHECK OPTION` rechaza cualquier `INSERT` o `UPDATE` hecho por la vista que deje una fila fuera de ella (por ejemplo, cambiar el género a novela). Sin esa opción, la fila se escribe y «desaparece» de la vista.

Otras diferencias. Para cambiar una vista, MySQL tiene `CREATE OR REPLACE VIEW`; SQLite no, y hay que hacer `DROP VIEW v_poesia;` y crearla de nuevo. Ninguno de los dos tiene vistas *materializadas* (que guardan el resultado y se refrescan); PostgreSQL y Oracle sí.

10. Ejemplo resuelto · El lector de cada comuna

El problema. La biblioteca arma un consejo de lectores con un representante por comuna: el socio que más libros ha pedido en su comuna. La encargada quiere poder repetir la consulta cada mes sin escribirla de nuevo.

Paso 1 · En palabras. Para cada socio, cuántos préstamos tiene (incluidos los que tienen cero). Después, en cada comuna, el que tiene el máximo. Son dos agregaciones: contar por socio y tomar el máximo por comuna, con la condición de que el máximo sea **de su propia comuna**. Eso pide una subconsulta correlacionada sobre una tabla de conteos.

Paso 2 · La tabla de conteos, guardada como vista. Se parte de `socio` con `LEFT JOIN` para no perder a quien no ha pedido nada, y se cuenta `p.id_prestamo` (que es `NULL` para ellos, así que cuenta 0). Todas las columnas del `SELECT` que no son agregados van en el `GROUP BY`:

```
CREATE VIEW v_prestamos_por_socio AS
SELECT s.id_socio, s.comuna, s.nombre, COUNT(p.id_prestamo) AS n
FROM socio s
LEFT JOIN prestamo p ON p.id_socio = s.id_socio
GROUP BY s.id_socio, s.comuna, s.nombre;
```

```
SELECT comuna, nombre, n
FROM v_prestamos_por_socio
ORDER BY comuna, n DESC;
```

comuna	nombre	n
Coelemu	Hugo Leiva	1
Coelemu	Felipe Araya	0
Penco	Carla Muñoz	4
Penco	Elisa Vera	2
Tomé	Ana Soto	4
Tomé	Benjamín Rojas	2
Tomé	Gabriela Paz	2
Tomé	Diego Fuentes	0

Paso 3 · El máximo de su comuna. Para cada fila `x`, la subconsulta busca el máximo de `n` entre las filas `y` de la **misma comuna**:

```
SELECT comuna, nombre, n
FROM v_prestamos_por_socio x
WHERE n = (SELECT MAX(y.n) FROM v_prestamos_por_socio y WHERE y.comuna = x.comuna)
ORDER BY comuna;
```

comuna	nombre	n
Coelemu	Hugo Leiva	1
Penco	Carla Muñoz	4
Tomé	Ana Soto	4

Fila por fila: para Felipe (Coelemu, 0) el máximo de Coelemu es 1, no pasa; para Hugo (Coelemu, 1), pasa. Para Benjamín (Tomé, 2) el máximo de Tomé es 4, no pasa; para Ana, sí.

Paso 4 · Lo mismo sin vista. Si la encargada no tuviera permiso para crear vistas, la tabla de conteos va en un `WITH` y la consulta queda autocontenida. El resultado es idéntico:

```
WITH por_socio AS (
    SELECT s.id_socio, s.comuna, s.nombre, COUNT(p.id_prestamo) AS n
    FROM socio s
    LEFT JOIN prestamo p ON p.id_socio = s.id_socio
    GROUP BY s.id_socio, s.comuna, s.nombre
)
SELECT comuna, nombre, n
FROM por_socio x
WHERE n = (SELECT MAX(y.n) FROM por_socio y WHERE y.comuna = x.comuna)
ORDER BY comuna;
```

comuna	nombre	n
Coelemu	Hugo Leiva	1
Penco	Carla Muñoz	4
Tomé	Ana Soto	4

Cómo se lee.

- **El consejo** lo forman Ana Soto (Tomé), Carla Muñoz (Penco) y Hugo Leiva (Coelemu).
- **Los empates entran todos.** Si Elisa hubiera tenido 4 préstamos, la consulta devolvería dos filas para Penco. No es un error: la regla «el que más pide» no dice qué hacer con un empate, y la consulta lo muestra en vez de esconderlo. Si hay que elegir uno, se agrega el criterio (el que pidió primero, por ejemplo).
- **El dato más útil no es el ganador.** El mejor lector de Coelemu tiene un solo préstamo. La decisión que sale de esta tabla es otra: Coelemu necesita difusión antes que un representante.
- **La vista se queda.** El próximo mes la encargada corre el paso 3 tal cual y la vista ya incluye los préstamos nuevos.

11. Errores típicos

Error	Cómo evitarlo
Agregado en el `WHERE`: `WHERE dias > AVG(dias)` (SQLite: `misuse of aggregate function`)	el agregado va en una subconsulta: `WHERE dias > (SELECT AVG(dias) FROM prestamo)`
`columna.alias` en vez de `alias.columna`, o el alias pegado a la tabla con punto (`FROM empleados.e`)	se declara `FROM empleados e` (o `AS e`) y se usa `e.sueldo`: primero la tabla o su alias, después la columna
`NOT IN (SELECT col ...)` con `NULL` en `col`: devuelve cero filas sin avisar	`NOT EXISTS`, o `WHERE col IS NOT NULL` dentro de la subconsulta
Comparar con `= NULL` en vez de `IS NULL` dentro de la subconsulta	`p.dias IS NULL`; `= NULL` nunca es verdadero
Subconsulta con `=` que devuelve varias filas: SQLite toma la primera en silencio, MySQL da error 1242	si puede haber varias, es `IN`; si debe haber una, revisa la condición
Olvidar la correlación (`WHERE s2.id_producto = s.id_producto`): compara contra el mínimo de **toda** la tabla	pregúntate «¿el máximo de qué grupo?» y escribe ese grupo en la subconsulta
Tabla derivada sin alias: corre en SQLite, falla en MySQL (error 1248)	ponle siempre `AS x`
Promediar sobre la población equivocada (los que tienen filas, no todos)	decide de qué tabla partes; `LEFT JOIN` si los que tienen cero cuentan
`UNION` cuando se iba a sumar: los duplicados se pierden	`UNION ALL` para sumar o contar; `UNION` para una lista sin repetidos
Creer que la vista es una copia de los datos	la vista es una consulta guardada: se recalcula cada vez
Escribir sobre una vista con `GROUP BY`, o sobre cualquier vista en SQLite	se escribe en la tabla base; solo las vistas simples de una tabla son actualizables (y en MySQL)

12. Práctica

Diez ejercicios: 11.1 a 11.4 básicos, 11.5 a 11.8 intermedios, 11.9 y 11.10 de desafío. Los que dicen «biblioteca» usan la base de la sección 1 (con las vistas `v_prestamo` y `v_poesia` ya creadas); los que dicen «caso» usan una copia de `datos/estampa_sur.db`. Escribe y corre cada consulta antes de mirar la pauta, que está en la sección 13 con el resultado real.

11.1 · Productos sobre el promedio (básico, caso). ¿Qué productos de Estampa Sur tienen un precio de lista mayor que el precio promedio de los cinco? Muestra nombre y precio, del más caro al más barato, y di cuánto es el promedio.

11.2 · Quién compra gorros (básico, caso). (a) Con `IN`, ¿cuántos clientes han comprado gorros (`id_producto = 5`) alguna vez? (b) Con `NOT IN`, ¿cuántos no? (c) ¿Por qué (a) y (b) suman 30 y no 26, que es el número de clientes con pedidos? (d) ¿Corre acá la trampa del `NULL`?

11.3 · Libros afuera (básico, biblioteca). (a) Con `EXISTS`, los títulos de los libros que hoy están prestados (algún préstamo con `dias` en `NULL`), en orden alfabético. (b) Un compañero escribió lo mismo con `p.dias = NULL`. ¿Qué devuelve y por qué?

11.4 · Comunas y géneros (básico, biblioteca). Usando la vista `v_prestamo`: (a) las comunas donde se ha pedido un libro infantil pero ninguna novela; (b) las comunas donde se han pedido las dos cosas. ¿Qué operador de conjuntos usa cada una?

11.5 · El pedido más grande de cada tipo de cliente (intermedio, caso). Para cada tipo de cliente (colegio, club, empresa, persona), el pedido de mayor monto (suma de `cantidad * precio_unit` de sus líneas): tipo, número de pedido y monto. Escríbelo con un `WITH` que calcule el total de cada pedido y una subconsulta correlacionada.

11.6 · Corrige la consulta (intermedio, caso). Un estudiante quiere los pedidos cuyo monto supera el monto promedio por pedido y escribe:

```
SELECT id_pedido
FROM detalle_pedido
WHERE SUM(cantidad * precio_unit) > AVG(SUM(cantidad * precio_unit))
GROUP BY id_pedido;
```

(a) ¿Qué responde SQLite y por qué? (b) Reescríbela con `WITH` y cuenta cuántos pedidos superan el promedio; informa también el promedio. (c) Escríbela de nuevo con `HAVING` y una tabla derivada, y comprueba que da lo mismo.

11.7 · Una vista de ventas por cliente (intermedio, caso). (a) Crea la vista `v_ventas_cliente` con id, nombre, tipo y monto total comprado por cliente. (b) Con ella, los clientes que han comprado más de $3.000.000. (c) ¿Cuántas filas tiene la vista y por qué no son 30? ¿Qué cambiarías para que estén los 30?

11.8 · El préstamo más largo de cada socio (intermedio, biblioteca). (a) Para cada socio que ha devuelto algo, el préstamo en que tuvo el libro más días: socio, préstamo y días. (b) Antes de correrla, predice qué devuelve si se olvida la condición que correlaciona la subconsulta. Córrela y compara.

11.9 · ¿Se puede escribir por la vista? (desafío, biblioteca). Hay tres vistas: `v_poesia` (sección 9), `v_prestamo` (sección 9) y esta:

```
CREATE VIEW v_resumen_genero AS
SELECT genero, COUNT(*) AS prestamos
FROM v_prestamo
GROUP BY genero;
```

(a) Para cada una, di si en MySQL se podría hacer un `UPDATE` por ella y por qué. (b) ¿Qué responde SQLite a `INSERT INTO v_poesia (id_libro, titulo, anio) VALUES (10, 'Versos del Itata', 2019);`? (c) En MySQL ese mismo `INSERT` también fallaría, por otra razón: ¿cuál? (d) ¿Qué evitaría `WITH CHECK OPTION` en `v_poesia`?

11.10 · Comprar las prendas de la semana 6 (desafío, caso). La semana 6 del programa maestro tiene un lote de cada producto (tabla `orden_produccion`). Estampa Sur compra cada prenda lisa al proveedor más barato que la ofrece (tabla `suministra`). (a) Para cada producto de la semana 6: proveedor elegido, lote,

costo por unidad y costo del lote. (b) Cuánto se le compra a cada proveedor esa semana, con un `WITH`. (c) Cuánto costaría la semana si se le comprara siempre al más caro, y cuánto se ahorra eligiendo al más barato.

---

13. Pautas

11.1.

```
SELECT AVG(precio) AS precio_promedio FROM producto;
```

precio_promedio
5170.0

```
SELECT nombre, precio
FROM producto
WHERE precio > (SELECT AVG(precio) FROM producto)
ORDER BY precio DESC;
```

nombre	precio
polerones	9890
poleras estampado doble	6290

El promedio es $5.170. Solo dos de los cinco productos están sobre él: el polerón lo tira hacia arriba, y por eso tres productos quedan bajo el promedio.

11.2. (a)

```
SELECT COUNT(*) AS clientes
FROM cliente
WHERE id_cliente IN (SELECT p.id_cliente
                     FROM pedido p
                     JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
                     WHERE d.id_producto = 5);
```

clientes
22

(b)

```
SELECT COUNT(*) AS clientes
FROM cliente
WHERE id_cliente NOT IN (SELECT p.id_cliente
                         FROM pedido p
                         JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
                         WHERE d.id_producto = 5);
```

clientes
8

(c) La consulta de afuera recorre los 30 clientes de la tabla `cliente`; cada uno está o no está en la lista. Entre los 8 que no compraron gorros están los 4 clientes sin ningún pedido. (d) No: la lista sale de `pedido.id_cliente`, que es `NOT NULL`. Por eso `NOT IN` es seguro acá.

11.3. (a)

```
SELECT titulo
FROM libro l
WHERE EXISTS (SELECT 1 FROM prestamo p WHERE p.id_libro = l.id_libro AND p.dias IS NULL)
ORDER BY titulo;
```

titulo
Hijo de ladrón
La casa de los espíritus
Mala onda

(b) Con `= NULL` no devuelve nada:

```
SELECT titulo
FROM libro l
WHERE EXISTS (SELECT 1 FROM prestamo p WHERE p.id_libro = l.id_libro AND p.dias = NULL);
```

titulo

`p.dias = NULL` es desconocido para toda fila (también cuando `dias` es `NULL`), así que la subconsulta nunca encuentra filas y `EXISTS` es siempre falso. Los tres libros siguen afuera: la biblioteca tiene que llamar a quienes los tienen.

11.4. (a) Diferencia, con `EXCEPT`:

```
SELECT comuna FROM v_prestamo WHERE genero = 'infantil'
EXCEPT
SELECT comuna FROM v_prestamo WHERE genero = 'novela';
```

comuna
Coelemu

(b) Intersección, con `INTERSECT`:

```
SELECT comuna FROM v_prestamo WHERE genero = 'infantil'
INTERSECT
SELECT comuna FROM v_prestamo WHERE genero = 'novela';
```

comuna
Penco

En Coelemu se piden libros infantiles y nunca novelas; en Tomé, al revés (novelas y nada infantil), por eso no sale en ninguna de las dos.

11.5.

```
WITH total_pedido AS (
    SELECT p.id_pedido, c.tipo, SUM(d.cantidad * d.precio_unit) AS total
    FROM pedido p
    JOIN cliente c        ON c.id_cliente = p.id_cliente
    JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
    GROUP BY p.id_pedido, c.tipo
)
SELECT tipo, id_pedido, total
FROM total_pedido t
WHERE total = (SELECT MAX(t2.total) FROM total_pedido t2 WHERE t2.tipo = t.tipo)
ORDER BY total DESC;
```

tipo	id_pedido	total
colegio	36	1216130
empresa	63	1129600
club	32	1010360
persona	85	86190

El pedido más grande de un colegio supera $1,2 millones; el de una persona no llega a $90.000. Un pedido de colegio vale lo que más de una docena de pedidos de personas: es el tipo de cliente que conviene atender primero en el taller.

11.6. (a) SQLite responde:

```
misuse of aggregate function SUM()
```

El `WHERE` se evalúa fila por fila, antes de agrupar: ahí todavía no existe la suma de un pedido, y menos el promedio de las sumas. (b)

```
WITH total_pedido AS (
    SELECT id_pedido, SUM(cantidad * precio_unit) AS total
    FROM detalle_pedido
    GROUP BY id_pedido
)
SELECT COUNT(*) AS pedidos, ROUND((SELECT AVG(total) FROM total_pedido), 2) AS promedio
FROM total_pedido
WHERE total > (SELECT AVG(total) FROM total_pedido);
```

pedidos	promedio
48	297695.33

(c)

```
SELECT COUNT(*) AS pedidos
FROM (SELECT id_pedido
      FROM detalle_pedido
      GROUP BY id_pedido
      HAVING SUM(cantidad * precio_unit) > (SELECT AVG(t.total)
                                            FROM (SELECT SUM(cantidad * precio_unit) AS
total
                                                  FROM detalle_pedido
                                                  GROUP BY id_pedido) AS t));
```

pedidos
48

El pedido promedio es de $297.695. Solo 48 de los 120 pedidos lo superan: unos pocos pedidos grandes (de colegios, clubes y empresas) suben el promedio, y la mayoría queda bajo él.

11.7. (a)

```
CREATE VIEW v_ventas_cliente AS
SELECT c.id_cliente, c.nombre, c.tipo, SUM(d.cantidad * d.precio_unit) AS total
FROM cliente c
JOIN pedido p         ON p.id_cliente = c.id_cliente
JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
GROUP BY c.id_cliente, c.nombre, c.tipo;
```

(b)

```
SELECT nombre, tipo, total
FROM v_ventas_cliente
WHERE total > 3000000
ORDER BY total DESC;
```

nombre	tipo	total
Cliente 10	club	3852310
Cliente 03	colegio	3114770
Cliente 12	empresa	3107770

(c)

```
SELECT COUNT(*) AS filas FROM v_ventas_cliente;
```

filas
26

Los 4 clientes sin pedidos no tienen filas con qué hacer el `JOIN` y desaparecen. Para que estén los 30, la vista parte de `cliente` con `LEFT JOIN` a `pedido` y a `detalle_pedido`, y el total se escribe `COALESCE(SUM(d.cantidad * d.precio_unit), 0)` para que diga 0 en vez de `NULL`.

11.8. (a)

```
SELECT p.id_socio, p.id_prestamo, p.dias
FROM prestamo p
WHERE p.dias = (SELECT MAX(p2.dias) FROM prestamo p2 WHERE p2.id_socio = p.id_socio)
ORDER BY p.id_socio;
```

id_socio	id_prestamo	dias
1	1	12
2	3	20
3	5	15
5	13	8
7	9	18
8	14	3

(b) Sin la correlación, la subconsulta calcula el máximo de **todos** los préstamos (20 días) y solo queda el préstamo que llegó a 20:

```
SELECT p.id_socio, p.id_prestamo, p.dias
FROM prestamo p
WHERE p.dias = (SELECT MAX(p2.dias) FROM prestamo p2)
ORDER BY p.id_socio;
```

id_socio	id_prestamo	dias
2	3	20

El síntoma de una correlación olvidada es este: la pregunta era «de cada uno» y sale una sola fila.

11.9. (a) `v_poesia`: sí. Es una sola tabla, con columnas tal cual y un `WHERE`; cada fila de la vista es una fila de `libro`. `v_prestamo`: solo en parte. Es un `JOIN` de tres tablas; MySQL deja actualizar una columna que venga de una sola de ellas (cambiar `p.dias`, por ejemplo), pero no insertar filas nuevas ni tocar columnas de dos tablas a la vez. `v_resumen_genero`: no. Cada fila resume varios préstamos con `GROUP BY` y `COUNT(*)`; poner `prestamos = 10` no dice qué préstamos crear o borrar. (b) SQLite no deja escribir en ninguna vista:

```
INSERT INTO v_poesia (id_libro, titulo, anio) VALUES (10, 'Versos del Itata', 2019);
```

```
cannot modify v_poesia because it is a view
```

(c) La vista no tiene `autor` ni `genero`, que en `libro` son `NOT NULL` sin valor por defecto: MySQL no tiene qué poner ahí y rechaza la fila. Y aunque tuviera las columnas, el libro nuevo no tendría `genero = 'poesía'`, así que no se vería en la vista. (d) `WITH CHECK OPTION` rechaza cualquier escritura por la vista que deje la fila fuera de ella: insertar un libro que no sea de poesía, o cambiarle el género a uno de poesía. Evita que alguien «pierda» filas escribiendo por una vista.

11.10. (a)

```
SELECT pr.nombre, pv.nombre AS proveedor, o.lote, s.costo_unidad, o.lote * s.costo_unidad
AS costo_lote
FROM orden_produccion o
JOIN producto pr  ON pr.id_producto = o.id_producto
JOIN suministra s ON s.id_producto = o.id_producto
JOIN proveedor pv ON pv.id_proveedor = s.id_proveedor
WHERE o.semana = 6
  AND s.costo_unidad = (SELECT MIN(s2.costo_unidad) FROM suministra s2 WHERE s2.id_producto
= s.id_producto)
ORDER BY o.id_producto;
```

nombre	proveedor	lote	costo_unidad	costo_lote
poleras básicas	Telas Norte	720	2400	1728000
polerones	Algodones Sur	360	5400	1944000
poleras estampado doble	Textil Ñuble	360	3600	1296000
bolsas	Telas Norte	540	1350	729000
gorros	Textil Ñuble	480	1880	902400

(b)

```
WITH compra AS (
    SELECT s.id_proveedor, o.lote * s.costo_unidad AS costo_lote
    FROM orden_produccion o
    JOIN suministra s ON s.id_producto = o.id_producto
    WHERE o.semana = 6
      AND s.costo_unidad = (SELECT MIN(s2.costo_unidad) FROM suministra s2 WHERE
s2.id_producto = s.id_producto)
)
SELECT pv.nombre AS proveedor, SUM(c.costo_lote) AS total
FROM compra c
JOIN proveedor pv ON pv.id_proveedor = c.id_proveedor
GROUP BY pv.nombre
ORDER BY total DESC;
```

proveedor	total
Telas Norte	2457000
Textil Ñuble	2198400
Algodones Sur	1944000

(c)

```
SELECT SUM(o.lote * (SELECT MIN(s.costo_unidad) FROM suministra s WHERE s.id_producto =
o.id_producto)) AS al_mas_barato,
       SUM(o.lote * (SELECT MAX(s.costo_unidad) FROM suministra s WHERE s.id_producto =
o.id_producto)) AS al_mas_caro
FROM orden_produccion o
WHERE o.semana = 6;
```

al_mas_barato	al_mas_caro
6599400	6635400

La semana 6 compra $6.599.400 en prendas lisas: $2.457.000 a Telas Norte, $2.198.400 a Textil Ñuble y $1.944.000 a Algodones Sur. Comprarle siempre al más caro costaría $6.635.400; elegir al más barato ahorra $36.000, poco, porque solo las poleras básicas y los gorros tienen dos proveedores ($30 de diferencia por unidad en cada uno). El ahorro real de tener dos proveedores no está en el precio sino en no depender de uno solo.

---

14. Caso integrador · Estampa Sur pregunta en dos pasos

> Este caso sigue a Estampa Sur por todo el ramo. La unidad 10 respondió preguntas de un paso (ventas por tienda, margen por tipo de cliente, clientes sin pedidos). Acá vienen las que necesitan una consulta dentro de otra. Todo corre sobre una copia de `datos/estampa_sur.db` con `sql/u11_caso.sql`.

Clientes que compran sobre el promedio. Es la consulta `u11_clientes_sobre_promedio` del script del caso, tal cual:

```
SELECT COUNT(*) FROM (
    SELECT p.id_cliente, SUM(d.cantidad * d.precio_unit) AS total
    FROM pedido p JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
    GROUP BY p.id_cliente) x
WHERE x.total > (SELECT AVG(t) FROM (
    SELECT SUM(d.cantidad * d.precio_unit) AS t
    FROM pedido p JOIN detalle_pedido d ON d.id_pedido = p.id_pedido GROUP BY p.id_cliente)
y);
```

COUNT(*)
12

Se lee de adentro hacia afuera: la tabla derivada `x` tiene el total de cada cliente con pedidos; la subconsulta escalar promedia esos totales; el `WHERE` deja los que lo superan. Las dos tablas derivadas tienen alias (`x` e `y`). La primera versión del script dejaba la segunda sin nombre: SQLite la aceptaba y MySQL respondía el error 1248 de la sección 6. Por eso se escribe `) y)` al final, aunque `y` no se use en ninguna otra parte.

¿Promedio de cuántos clientes?

```
SELECT ROUND(AVG(t), 2) AS promedio, COUNT(*) AS clientes
FROM (SELECT SUM(d.cantidad * d.precio_unit) AS t
      FROM pedido p JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
      GROUP BY p.id_cliente);
```

promedio	clientes
1373978.46	26

El cliente promedio compró $1.373.978 en marzo, y el promedio es de los 26 que compraron: los 4 sin pedidos no entran. La misma pregunta con `WITH`, más fácil de leer y sin repetir la subconsulta:

```
WITH total_cliente AS (
    SELECT p.id_cliente, SUM(d.cantidad * d.precio_unit) AS total
    FROM pedido p
    JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
    GROUP BY p.id_cliente
)
SELECT COUNT(*) AS clientes
FROM total_cliente
WHERE total > (SELECT AVG(total) FROM total_cliente);
```

clientes
12

Doce de 26 clientes compran sobre el promedio, y los doce son colegios, clubes o empresas (hay 18 de esos). Ninguna de las 8 personas naturales con pedidos llega al promedio: compran de a pocas unidades.

El proveedor más barato de cada producto. Consulta `u11_proveedor_mas_barato`, una subconsulta correlacionada:

```
SELECT pr.nombre AS producto, pv.nombre AS proveedor, s.costo_unidad
FROM suministra s
JOIN producto pr  ON pr.id_producto = s.id_producto
JOIN proveedor pv ON pv.id_proveedor = s.id_proveedor
WHERE s.costo_unidad = (SELECT MIN(s2.costo_unidad) FROM suministra s2 WHERE s2.id_producto
= s.id_producto)
ORDER BY pr.id_producto;
```

producto	proveedor	costo_unidad
poleras básicas	Telas Norte	2400
polerones	Algodones Sur	5400
poleras estampado doble	Textil Ñuble	3600
bolsas	Telas Norte	1350
gorros	Textil Ñuble	1880

Los `AS producto` y `AS proveedor` no son adorno: `producto` y `proveedor` tienen las dos una columna `nombre`, y sin alias el resultado traería dos columnas llamadas `nombre` (así estaba la primera versión del script). SQLite lo acepta, pero un programa que lea el resultado por nombre de columna toma la que no era. Si se olvida la correlación, la subconsulta da el mínimo de toda la tabla ($1.350) y solo sobrevive una fila:

```
SELECT pr.nombre AS producto, pv.nombre AS proveedor, s.costo_unidad
FROM suministra s
JOIN producto pr  ON pr.id_producto = s.id_producto
JOIN proveedor pv ON pv.id_proveedor = s.id_proveedor
WHERE s.costo_unidad = (SELECT MIN(s2.costo_unidad) FROM suministra s2)
ORDER BY pr.id_producto;
```

producto	proveedor	costo_unidad
bolsas	Telas Norte	1350

La producción de la semana 6. Consulta `u11_produccion_semana_6`:

```
SELECT pr.nombre, o.lote, o.lote * pr.seg_estampado / 3600.0 AS horas_estampado
FROM orden_produccion o JOIN producto pr ON pr.id_producto = o.id_producto
WHERE o.semana = 6 ORDER BY o.id_producto;
```

nombre	lote	horas_estampado
poleras básicas	720	6.0
polerones	360	9.0
poleras estampado doble	360	11.0
bolsas	540	3.0
gorros	480	4.0

```
SELECT SUM(o.lote) AS prendas, SUM(o.lote * pr.seg_estampado) / 3600.0 AS horas_estampado
FROM orden_produccion o JOIN producto pr ON pr.id_producto = o.id_producto
WHERE o.semana = 6;
```

prendas	horas_estampado
2460	33.0

El `3600.0` (con punto decimal) importa: con `3600` la división sería entera en SQLite y truncaría. Son las mismas 2.460 prendas y 33 horas de estampado que el programa maestro de Gestión de Operaciones (unidad 7 de ese ramo) dejó en la semana 6: la base guarda el plan que allá se calculó a mano.

Una vista de ventas por tienda. La consulta de ventas por tienda de la unidad 10 se va a usar todo el tiempo (en el resto de esta unidad, en la 12 y en cualquier informe). Queda como vista:

```
CREATE VIEW v_ventas_tienda AS
SELECT t.nombre AS tienda, SUM(d.cantidad * d.precio_unit) AS ventas
FROM detalle_pedido d
JOIN pedido p  ON p.id_pedido = d.id_pedido
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN tienda t  ON t.id_tienda = c.id_tienda
GROUP BY t.nombre;
```

```
SELECT tienda, ventas FROM v_ventas_tienda ORDER BY ventas DESC;
```

tienda	ventas
Valle	7297650
Centro	6951140
Lagos	6134350
Costa	5922540
Puerto	5534370
Cerro	3883390

Son las cifras de la unidad 10: Valle $7.297.650, Centro $6.951.140, Lagos $6.134.350, Costa $5.922.540, Puerto $5.534.370 y Cerro $3.883.390. La vista no tiene `ORDER BY`: el orden se pide al consultarla. Y como es una tabla más, admite subconsultas. Las tiendas sobre el promedio:

```
SELECT tienda, ventas
FROM v_ventas_tienda
WHERE ventas > (SELECT AVG(ventas) FROM v_ventas_tienda)
ORDER BY ventas DESC;
```

tienda	ventas
Valle	7297650
Centro	6951140
Lagos	6134350

La tienda promedio vendió $5.953.907 en marzo; Costa queda apenas debajo y Cerro, lejos. Es el mismo patrón de los clientes: la vista se escribe una vez y cada pregunta nueva es una línea.

La cadena. El verificador crea la base del caso en una carpeta temporal con el mismo script (la base de `datos/` no se toca), corre `u11_caso.sql` y exige que las tres consultas `u11_*` sean las del script, que den lo mismo (12 clientes, los cinco proveedores, las 33 horas) y que la vista dé las ventas por tienda de la unidad 10 (`CADENA: OK`).

Lo que pasa a la unidad 12. Hasta acá la base solo se ha leído. La unidad 12 registra un pedido nuevo como una transacción y mira cómo cambia `v_ventas_tienda` sin tocarla; después le pone índices a la consulta de ventas de una tienda y compara su plan antes y después.

---

15. Para seguir

En los libros de referencia del área. El programa del ramo no fija bibliografía. Elmasri y Navathe, *Fundamentos de sistemas de bases de datos*, y Silberschatz, Korth y Sudarshan, *Fundamentos de bases de datos*, tratan subconsultas, consultas anidadas correlacionadas y vistas en sus capítulos de SQL.

Abierto y gratis. La documentación de SQLite, en inglés, es corta y precisa:

- *The WITH Clause*, con el `WITH` y las CTE: https://www.sqlite.org/lang_with.html
- *CREATE VIEW*, con lo que SQLite permite y no permite en una vista: https://www.sqlite.org/lang_createview.html

Con el tutor. La carpeta `tutor/` trae el contexto de este ramo. Pégalo al inicio de la conversación con cualquier IA y pídele, por ejemplo: «Dame una base chica con una columna que tenga `NULL` y tres preguntas que se respondan con `NOT IN` o `NOT EXISTS`; no me digas cuál cae en la trampa hasta que yo responda».

Base abierta de ingeniería · Camilo Merino · Bases de Datos · versión 0.1 (16-10-2026). Ejercicios y datos originales; las cifras se verifican con el script de la unidad.